```
                {                                                          ❺
                    return new ValidationResult(ErrorMessage               ❹
                        ?? $"The file extension must be                    ❹
                        ➥ {Extensions}");                                 ❹
                }                                                          ❹
            }
            return ValidationResult.Success;                               ❺
        }
    }
}
```

❶ Create a private field to house the allowed extensions.

❷ Create a public property, so the user of the attribute can set the allowed extensions.

❸ Assign the user-supplied extensions to the private field, converting them from a comma-separated string to a collection of lowercase values.

❹ You use pattern matching to ensure the property this attribute has been applied to is an IFormFile. If it is, and the user has provided extensions, you check to see if the filename matches any of them. If not, you return an error message.

❺ In all other cases, validation succeeds.

This is not a production-ready validator. It doesn't, for example, take into account users not knowing whether to include the leading dot in the list of allowed extensions. However, it will suffice for demonstration. It is very similar to the previous validator in that it provides a public property enabling the user to pass in a list of accepted file extensions. Suppose the property the attribute has been applied to is an `IFormFile` type. In that case, the validation logic is executed, and if the file extension has not been provided, the validation fails.

Now you need to add a page for creating cities. Add a new folder to the Pages folder named CityManager, and within that, add a new Razor page named Create. Open the Create.cshtml.cs file, and change the `using` directives to the following.

Listing 6.25 Using directives for the city's `CreateModel` class